

LAPORAN PROYEK
MATA KULIAH ARTIFICIAL INTELLIGENCE

**ANALISIS KOMPARATIF
DYNAMIC A* WITH SCENARIO ENGINE**

Sistem Navigasi Jalur Terpendek Adaptif untuk Kampus UNIB



Disusun oleh:

Ali Akbar Bermano

G1A024041

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

Program Studi Informatika

Fakultas Teknik

Universitas Bengkulu

Semester Genap 2025/2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Permasalahan pencarian jalur terpendek (shortest path problem) merupakan salah satu topik fundamental dalam ilmu komputer dan kecerdasan buatan. Dalam konteks nyata, seperti navigasi di lingkungan kampus Universitas Bengkulu (UNIB), permasalahan ini tidak hanya melibatkan jarak fisik antar titik, tetapi juga faktor dinamis seperti kondisi permukaan jalan, kepadatan lalu lintas, dan penutupan ruas tertentu yang dapat berubah sewaktu-waktu.

Proyek Shortest-Path Dynamic A* Engine for the UNIB Campus Graph mengimplementasikan algoritma Dynamic A* sebagai mesin pencarian jalur adaptif. Algoritma ini dipilih karena kemampuannya menggabungkan informasi biaya aktual (g-score) dengan estimasi heuristik (h-score) untuk memandu pencarian secara terarah, sehingga secara teoritis lebih efisien dibandingkan algoritma pencarian graf konvensional seperti Dijkstra dan Breadth-First Search (BFS).

Dokumen ini menyajikan analisis mendalam terhadap algoritma A* yang digunakan dalam proyek tersebut, mencakup cara kerja, kelebihan, dan keterbatasannya; peran fungsi heuristik Haversine dalam menentukan efisiensi pencarian; serta perbandingan komparatif berbasis metrik formal terhadap algoritma Dijkstra dan BFS. Analisis ini bertujuan memberikan justifikasi ilmiah atas pemilihan A* sebagai algoritma inti sistem navigasi kampus UNIB.

1.2 Rumusan Masalah

- Bagaimana mekanisme kerja algoritma Dynamic A* dalam konteks graf berbobot dinamis kampus UNIB?
- Apa peran fungsi heuristik Haversine dalam menentukan efisiensi dan optimalitas pencarian rute?
- Bagaimana perbandingan kuantitatif dan kualitatif A* terhadap Dijkstra dan BFS dari sisi kompleksitas waktu, kompleksitas ruang, jumlah node yang dieksplorasi, dan kualitas rute yang dihasilkan?
- Dalam kondisi graf seperti apa A* lebih unggul atau kurang efektif dibandingkan Dijkstra?

1.3 Tujuan Analisis

- Menguraikan cara kerja, kelebihan, dan kekurangan algoritma Dynamic A* secara komprehensif.
- Menganalisis peran dan sifat matematis fungsi heuristik Haversine yang digunakan dalam proyek.
- Menyajikan perbandingan komparatif terstruktur antara A*, Dijkstra, dan BFS berdasarkan metrik ilmiah.

- Menentukan rekomendasi use case terbaik untuk setiap algoritma berdasarkan karakteristik graf.

1.4 Metodologi Analisis

Analisis dilakukan berdasarkan dua sumber utama: (1) kajian teoritis terhadap literatur algoritma pencarian graf, mencakup karya Hart, Nilsson, & Raphael (1968) untuk A*; Dijkstra (1959) untuk algoritma Dijkstra; serta teori BFS dalam Cormen et al. (2022); dan (2) analisis empiris terhadap implementasi aktual proyek Dynamic A* UNIB, merujuk pada kode sumber `algorithm.py`, `campus_data.py`, dan `web_server.py`, serta hasil pengujian sistem yang terdokumentasi dalam laporan proyek.

BAB II ANALISIS ALGORITMA A*

2.1 Cara Kerja Algoritma Dynamic A*

Algoritma A* (dibaca: A-star) pertama kali diperkenalkan oleh Hart, Nilsson, dan Raphael pada tahun 1968 sebagai generalisasi dari algoritma Dijkstra yang dipandu oleh fungsi heuristik. A* menggunakan fungsi evaluasi $f(n)$ untuk menentukan urutan eksplorasi node:

$$f(n) = g(n) + h(n)$$

Di mana:

- $g(n)$ adalah biaya aktual kumulatif dari node awal (start) ke node n , diperoleh melalui akumulasi fungsi `calc_cost()` pada setiap edge yang dilalui.
- $h(n)$ adalah fungsi heuristik yang mengestimasi biaya minimum dari node n ke node tujuan (goal). Dalam implementasi proyek ini, $h(n)$ dihitung menggunakan formula Haversine untuk mendapatkan jarak garis lurus antar koordinat GPS.
- $f(n)$ merupakan estimasi total biaya jalur melalui node n . Node dengan nilai $f(n)$ terkecil mendapat prioritas eksplorasi tertinggi.

Implementasi dalam proyek UNIB menggunakan variasi Dynamic A*, di mana bobot setiap edge dihitung secara real-time melalui fungsi `calc_cost()` dengan formula:

$$\text{cost}(e) = \text{distance} \times k_{\text{surface}} \times k_{\text{scenario}} \times k_{\text{condition}} \times \text{time_factor}$$

Pendekatan ini menjadikan A* bersifat 'dinamis' karena bobot graf tidak bersifat statis; setiap query rute dapat menghasilkan graf yang berbeda tergantung kondisi aktif saat itu.

2.1.1 Mekanisme Eksekusi `find_path()`

Proses eksekusi algoritma dalam fungsi `find_path()` berlangsung melalui tahapan berikut:

- Inisialisasi: Open list diinisialisasi sebagai min-heap (`heapq` Python) berisi tuple (f_score , $counter$, $node_id$). Closed set (visited) diinisialisasi kosong. $g_score[start] = 0$, $g_score[lainnya] = \infty$.
- Preprocessing: Semua bobot edge dihitung sekali di awal melalui `calc_cost()`. Heuristic_scale dihitung sebagai nilai minimum rasio $cost/distance$ seluruh edge aktif untuk menjaga admissibilitas heuristik meski ada edge VIP dengan pengali < 1 .
- Iterasi utama: Node dengan $f(n)$ terkecil di-pop dari heap. Jika sudah berada di closed set, dilewati (lazy deletion). Jika merupakan node tujuan, rekonstruksi jalur dimulai. Untuk setiap tetangga yang belum dikunjungi dan dapat dijangkau, tentative_g dihitung; jika lebih kecil dari g_score saat ini, g_score diperbarui dan node didorong ke heap.
- Rekonstruksi: Jalur direkonstruksi melalui kamus `came_from` dari goal menuju start secara mundur, kemudian dibalik menjadi urutan start→goal.
- Output: Fungsi mengembalikan path (daftar node ID), `total_dist_m` (jarak fisik), `total_cost` (biaya terakumulasi), `eta_minutes`, `iterations` (jumlah node yang diekstrak dari heap), dan `execution_ms`.

2.2 Analisis Fungsi Heuristik Haversine

Pemilihan heuristik merupakan keputusan desain kritis dalam implementasi A*. Proyek ini menggunakan formula Haversine sebagai fungsi heuristik $h(n)$, menggantikan jarak Euclidean sederhana yang umumnya digunakan pada koordinat Kartesius.

2.2.1 Formula Haversine

Formula Haversine menghitung jarak busur lingkaran besar (great-circle distance) antara dua titik di permukaan bumi berdasarkan koordinat lintang (latitude) dan bujur (longitude):

$$a = \sin^2(\Delta\text{lat}/2) + \cos(\text{lat}_1) \times \cos(\text{lat}_2) \times \sin^2(\Delta\text{lon}/2)$$

$$h(n) = 2R \times \arcsin(\sqrt{a}), \quad R = 6371 \text{ km}$$

Berbeda dari jarak Euclidean ($\sqrt{(\Delta x^2 + \Delta y^2)}$) yang mengabaikan kelengkungan bumi, Haversine memperhitungkan geometri bola sehingga menghasilkan estimasi yang lebih akurat untuk koordinat WGS-84 yang digunakan dalam sistem GPS.

2.2.2 Sifat Admissible dan Consistent

Agar A* terjamin menghasilkan jalur optimal, fungsi heuristik harus memenuhi dua syarat:

- **Admissible (dapat diterima):** $h(n)$ tidak pernah melebihi biaya aktual dari n ke tujuan. Haversine memenuhi syarat ini karena mengukur jarak garis lurus di permukaan bumi, yang secara geometris tidak mungkin lebih panjang dari jalur nyata yang harus melewati ruas-ruas jalan tertentu (sesuai prinsip ketidaksamaan segitiga).
- **Consistent (monoton):** $h(n) \leq \text{cost}(n, n') + h(n')$ untuk setiap edge (n, n') . Haversine memenuhi ini karena jarak garis lurus antara dua titik selalu $\leq$ jarak melalui titik ketiga.

Satu tantangan khusus muncul dalam proyek ini: adanya edge dengan pengali skenario $k_{\text{scenario}} < 1$ (kondisi VIP_ROUTE dengan koefisien 0.5), yang membuat cost aktual lebih kecil dari jarak fisik. Hal ini berpotensi melanggar admissibilitas heuristik Haversine yang berbasis jarak. Solusi yang diimplementasikan adalah menghitung heuristic_scale sebagai nilai minimum dari rasio ($\text{cost}/\text{distance}$) di seluruh edge aktif, lalu mengalikan $h(n)$ dengan skala tersebut, sehingga heuristik tetap bersifat admissible dalam semua kondisi.

2.2.3 Pengaruh Kualitas Heuristik terhadap Efisiensi

Kualitas heuristik sangat memengaruhi jumlah node yang perlu dieksplorasi oleh A*. Secara formal, semakin $h(n)$ mendekati biaya aktual $h^*(n)$ tanpa melampauinya, semakin sedikit node yang perlu dieksplorasi:

- $h(n) = 0$ untuk semua n : A* berperilaku identik dengan Dijkstra, mengeksplorasi semua node yang dapat dijangkau secara optimal berdasarkan $g(n)$ saja.
- $h(n) = h^*(n)$ untuk semua n (heuristik sempurna): A* hanya mengeksplorasi node-node yang benar-benar berada di jalur optimal, tanpa eksplorasi yang tidak perlu.
- $h(n) > h^*(n)$ (heuristik tidak admissible): A* dapat menghasilkan jalur suboptimal, tetapi berjalan lebih cepat (trade-off antara kecepatan dan optimalitas, digunakan dalam varian Weighted A*).

Haversine dalam konteks kampus UNIB menghasilkan heuristik yang berkualitas tinggi karena: (1) skala area kampus relatif kecil (jarak antar node berkisar 10–300 meter), sehingga perbedaan antara jarak garis lurus dan jalur aktual tidak ekstrem; dan (2) jaringan jalan kampus tidak memiliki bentuk geometri yang sangat tidak teratur (misalnya labirin), sehingga estimasi Haversine cukup akurat menggambarkan biaya aktual.

2.3 Kelebihan Algoritma Dynamic A*

2.3.1 Optimalitas Terjamin

Selama fungsi heuristik bersifat admissible, A* terjamin menghasilkan jalur dengan total biaya minimum. Dalam proyek UNIB, kondisi ini dipertahankan melalui mekanisme `heuristic_scale` yang menyesuaikan bobot heuristik secara dinamis berdasarkan pengali edge terendah yang aktif.

2.3.2 Efisiensi Pencarian yang Superior

Dibandingkan Dijkstra yang mengeksplorasi node secara radial ke semua arah, A* mengarahkan pencarian ke arah tujuan melalui panduan heuristik. Dalam graf kampus UNIB dengan ~220 node dan ~280 edge, sistem mencatat waktu eksekusi rata-rata di bawah 20ms, jauh lebih efisien daripada pencarian tanpa heuristik untuk target yang jauh dari sumber.

2.3.3 Adaptabilitas melalui Bobot Dinamis

Fungsi `calc_cost()` yang menghitung bobot secara real-time memungkinkan A* beradaptasi terhadap perubahan kondisi tanpa modifikasi pada struktur algoritma inti. Perubahan kondisi dari NORMAL ke CLOSED cukup mengubah nilai `k_condition` menjadi tak hingga, dan A* secara otomatis menghindari edge tersebut pada query berikutnya.

2.3.4 Fleksibilitas Skenario

Mekanisme Scenario Engine yang terintegrasi memungkinkan konfigurasi bobot dan pemblokiran edge secara batch tanpa mengubah logika algoritma. Fitur ini tidak tersedia secara inherent dalam Dijkstra atau BFS standar.

2.3.5 Penanganan Graf Tidak Terhubung

A* mengembalikan error `NO_PATH_FOUND` dengan mekanisme fallback ke skenario Normal ketika semua jalur ke tujuan terblokir oleh skenario aktif. Perilaku ini lebih informatif dibandingkan implementasi Dijkstra standar yang mengembalikan jarak tak hingga.

2.4 Kekurangan dan Keterbatasan Algoritma A*

2.4.1 Ketergantungan pada Kualitas Heuristik

Efisiensi A* sangat bergantung pada kualitas fungsi heuristik. Pada graf dengan geometri tidak teratur—misalnya labirin, jaringan jalan yang sangat berkelok, atau kondisi di mana jalur terpendek harus memutar jauh dari garis lurus—heuristik Haversine dapat memberikan estimasi

yang jauh lebih rendah dari biaya aktual, sehingga A* terpaksa mengeksplorasi lebih banyak node dan keunggulannya atas Dijkstra berkurang.

2.4.2 Overhead Komputasi Heuristik

Setiap iterasi A* memerlukan satu komputasi formula Haversine untuk setiap tetangga yang dievaluasi. Meskipun komputasi ini $O(1)$, pada graf dengan degree rata-rata tinggi dan ukuran sangat besar (jutaan node), overhead ini dapat menjadi faktor pembatas.

2.4.3 Konsumsi Memori Lebih Tinggi

A* mempertahankan open list (heap) dan closed set secara bersamaan, serta menyimpan `g_score` dan `came_from` untuk setiap node yang pernah dievaluasi. Dalam skenario terburuk (worst case), konsumsi memori A* setara dengan BFS pada graf yang sangat besar.

2.4.4 Tidak Optimal untuk Graf Negatif

A* standar tidak dapat menangani edge dengan bobot negatif, karena asumsi admissibilitas heuristik bergantung pada bobot non-negatif. Dalam proyek UNIB, hal ini bukan masalah karena semua biaya didefinisikan non-negatif ($k_{\text{surface}} \geq 1.0$, $k_{\text{condition}} \geq 1.0$ atau infinity), namun perlu diperhatikan jika sistem diperluas ke domain lain.

2.4.5 Bobot Diperbarui Secara Batch

Perubahan kondisi jalan yang terjadi saat pencarian sedang berlangsung tidak langsung memengaruhi jalur yang sedang dihitung. Sistem hanya memperbarui bobot pada awal setiap query baru. Untuk skenario dengan perubahan kondisi sangat dinamis (real-time per detik), diperlukan varian seperti D* Lite atau LPA* yang mendukung pembaruan inkremental.

BAB III

PERBANDINGAN KOMPARATIF ALGORITMA

3.1 Deskripsi Algoritma Pembanding

3.1.1 Algoritma Dijkstra

Algoritma Dijkstra (1959) menemukan jalur terpendek dari satu sumber ke semua node lain dalam graf berbobot non-negatif. Berbeda dari A*, Dijkstra tidak menggunakan heuristik; node dieksplorasi semata-mata berdasarkan $g(n)$ (biaya aktual dari sumber). Secara formal, ini setara dengan A* dengan $h(n) = 0$ untuk semua n . Dijkstra terjamin menghasilkan jalur optimal dan bekerja pada semua graf berbobot non-negatif, baik yang memiliki koordinat geografis maupun tidak.

3.1.2 Breadth-First Search (BFS)

BFS adalah algoritma pencarian graf yang mengeksplorasi node berdasarkan jarak hop (jumlah edge) dari sumber, menggunakan antrian FIFO. BFS tidak mempertimbangkan bobot edge—semua edge diperlakukan sama (bobot implisit = 1). BFS terjamin menemukan jalur dengan jumlah edge paling sedikit (bukan jalur dengan biaya minimum pada graf berbobot). Pada graf berbobot seperti jaringan jalan kampus UNIB, BFS umumnya tidak menghasilkan rute yang optimal dari sisi jarak atau waktu tempuh.

3.2 Perbandingan Kompleksitas

Tabel berikut menyajikan perbandingan formal kompleksitas waktu dan ruang ketiga algoritma. Notasi V menyatakan jumlah node, E jumlah edge, dan b branching factor rata-rata graf.

Metrik	A* Dinamis	Dijkstra	BFS
Kompleksitas Waktu (worst)	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V+E)$
Kompleksitas Waktu (praktis)	$O(E)$ hingga $O((V+E) \log V)$	$O((V+E) \log V)$	$O(V+E)$
Kompleksitas Ruang	$O(V)$, open list + closed set	$O(V)$, priority queue	$O(V)$, antrian FIFO
Struktur Data Inti	Min-heap + hash set	Min-heap + hash map	FIFO queue + visited set
Overhead per Iterasi	$g(n) + h(n) + \text{calc_cost}()$	$g(n)$ saja	Penghitungan hop saja
Waktu Eksekusi (UNIB, ~220 V)	< 20ms (terukur)	< 25ms (estimasi)	< 5ms (estimasi)

Tabel 3.1 ,Perbandingan kompleksitas teoritis ketiga algoritma

Meskipun A* dan Dijkstra memiliki kompleksitas asimptotik terburuk yang sama, $O((V+E) \log V)$ karena keduanya menggunakan min-heap, performa praktis A* biasanya jauh lebih baik. Hal ini disebabkan oleh fungsi heuristik yang menyebabkan A* mengeksplorasi jauh

lebih sedikit node sebelum menemukan tujuan, mereduksi konstanta tersembunyi dalam notasi O-besar secara signifikan.

3.3 Perbandingan Jumlah Node yang Dieksplorasi

Jumlah node yang dieksplorasi (explored nodes) merupakan metrik empiris yang lebih informatif daripada kompleksitas asimptotik dalam menilai performa algoritma pada instans spesifik. Tabel berikut menyajikan estimasi berdasarkan karakteristik graf kampus UNIB.

Skenario Graf	A*	Dijkstra	BFS
Graf kecil, tujuan dekat (< 50m)	1–5 node	5–15 node	3–10 node
Graf UNIB, tujuan sedang (100–500m)	10–40 node	40–120 node	80–180 node
Graf UNIB, tujuan jauh (> 500m)	30–80 node	100–200 node	150–220 node
Graf UNIB, ada edge ditutup (CLOSED)	30–90 node (reroute)	100–210 node	150–220+ node
Tujuan tidak dapat dicapai (no path)	~V node (semua)	~V node (semua)	~V node (semua)

Tabel 3.2 ,Estimasi jumlah node yang dieksplorasi per skenario (berdasarkan analisis graf UNIB)

Angka-angka pada Tabel 3.2 merupakan estimasi berdasarkan analisis struktur graf aktual UNIB (224 node aktif, ~280 edge). A* secara konsisten mengeksplorasi 60–80% lebih sedikit node dibandingkan Dijkstra pada skenario dengan tujuan di posisi tengah hingga jauh, karena heuristik Haversine secara efektif membatasi eksplorasi ke arah yang tidak relevan.

3.4 Perbandingan Kualitas Rute (Optimasi)

Kualitas rute merujuk pada kemampuan algoritma menghasilkan jalur dengan total biaya minimum sesuai fungsi objektif yang ditetapkan.

Aspek Kualitas	A* Dinamis	Dijkstra	BFS
Optimalitas pada graf berbobot	OPTIMAL*	OPTIMAL	TIDAK OPTIMAL
Optimalitas pada graf tidak berbobot	OPTIMAL	OPTIMAL	OPTIMAL (hop)
Penanganan bobot negatif	TIDAK	TIDAK	TIDAK RELEVAN
Adaptasi bobot dinamis	YA (calc_cost)	Butuh restart	TIDAK
Penanganan edge CLOSED (inf)	YA ,otomatis	YA ,manual	TIDAK RELEVAN
Mendukung skenario multi-kondisi	YA (Scenario Engine)	TIDAK	TIDAK
Akurasi pada graf berbobot nyata	Sangat Tinggi	Sangat Tinggi	Rendah

Tabel 3.3 ,Perbandingan kualitas rute dan fitur ketiga algoritma

* Tanda bintang menunjukkan optimalitas yang bergantung pada admissibilitas heuristik. Dalam implementasi UNIB, admissibilitas dijaga melalui mekanisme `heuristic_scale`.

3.5 Ringkasan Perbandingan Menyeluruh

Dimensi	A* Dinamis	Dijkstra	BFS	Catatan
Kompleksitas Waktu	$O((V+E) \log V)$	$O((V+E) \log V)$	$O(V+E)$	BFS tercepat secara teoritis; A* lebih cepat dalam praktik pada graf besar berbobot
Kompleksitas Ruang	$O(V)$	$O(V)$	$O(V)$	Ketiganya setara; A* menyimpan data lebih banyak per node
Node Dieksplorasi	Paling sedikit (praktis)	Sedang	Paling banyak	Keunggulan utama A* atas Dijkstra pada instans spesifik
Optimalitas Rute	Optimal*	Optimal	Tidak optimal (berbobot)	BFS optimal hanya pada graf tidak berbobot
Perlu Koordinat Geografis	YA (heuristik)	TIDAK	TIDAK	A* memerlukan informasi tambahan yang mungkin tidak selalu tersedia
Adaptasi Bobot Real-time	YA	TIDAK (perlu recompute)	TIDAK	Keunggulan kritis A* untuk navigasi dinamis
Kompleksitas Implementasi	Tinggi	Sedang	Rendah	Trade-off antara kemudahan implementasi dan performa
Cocok untuk UNIB Navigator	SANGAT COCOK	COCOK	TIDAK COCOK	Graf berbobot, berkoordinat, dengan kondisi dinamis

Tabel 3.4 ,Ringkasan perbandingan menyeluruh A*, Dijkstra, dan BFS

BAB IV

KASUS PENGGUNAAN TERBAIK (USE CASE ANALYSIS)

4.1 Kondisi di Mana A* Lebih Unggul dari Dijkstra

4.1.1 Graf dengan Koordinat Geografis yang Tersedia

Kondisi paling fundamental untuk keunggulan A* adalah ketersediaan informasi koordinat geografis (atau representasi spasial setara) yang memungkinkan konstruksi heuristik admissible yang bermakna. Jaringan jalan nyata, baik skala kampus, kota, maupun antar-kota, selalu memiliki koordinat ini. Dalam konteks UNIB, setiap node menyimpan koordinat GPS presisi WGS-84, menjadikan Haversine sebagai heuristik yang sangat akurat.

4.1.2 Pencarian Titik-ke-Titik (Single-Pair Shortest Path)

A* dirancang khusus untuk pencarian dari satu titik ke satu tujuan. Keunggulannya atas Dijkstra paling terasa ketika hanya diperlukan satu jalur, bukan jalur ke semua node. Pada skenario ini, heuristik secara aktif membatasi eksplorasi ke arah yang relevan, sehingga A* dapat berhenti segera setelah tujuan ditemukan tanpa harus mengeksplorasi seluruh komponen terhubung. Dijkstra, sebaliknya, terus mengeksplorasi node-node yang lebih dekat dari sumber meskipun tidak relevan dengan tujuan.

4.1.3 Graf Besar dengan Tujuan yang Jauh dari Sumber

Keunggulan A* atas Dijkstra makin signifikan seiring bertambahnya jarak antara sumber dan tujuan dalam graf. Saat jarak bertambah, Dijkstra harus mengeksplorasi semakin banyak node secara radial, sementara A* tetap terfokus pada arah tujuan. Untuk graf kampus UNIB dengan 224 node, keunggulan ini mungkin hanya menghemat beberapa milidetik, namun pada jaringan jalan kota dengan ratusan ribu node, penghematan ini dapat mencapai beberapa order of magnitude.

4.1.4 Sistem Navigasi Real-time dengan Bobot Dinamis

Kombinasi A* dengan Scenario Engine, sebagaimana diimplementasikan dalam proyek UNIB, menciptakan sistem yang tidak tersedia pada implementasi Dijkstra atau BFS standar: kemampuan beradaptasi terhadap perubahan kondisi graf (penutupan jalan, kepadatan, kondisi permukaan) sambil tetap menjamin optimalitas rute. Setiap query memanfaatkan kondisi terkini secara penuh tanpa perlu restrukturisasi algoritma.

4.1.5 Kebutuhan Waktu Respons Rendah pada Kueri Berulang

Dalam sistem navigasi web atau mobile yang harus merespons puluhan hingga ratusan kueri per menit, efisiensi A* yang mengeksplorasi lebih sedikit node memberikan keuntungan konkrit dalam hal throughput server. Data performa proyek UNIB menunjukkan rata-rata eksekusi di bawah 20ms per kueri pada graf aktual, memungkinkan sistem melayani banyak pengguna secara bersamaan.

4.2 Kondisi di Mana A* Kurang Efektif

4.2.1 Graf Tanpa Informasi Spasial

Bila graf tidak memiliki koordinat atau properti spasial yang dapat digunakan untuk membangun heuristik admissible yang bermakna, misalnya graf relasi sosial, graf ketergantungan perangkat lunak, atau jaringan komputer abstrak, heuristik yang tersedia biasanya $h(n) = 0$ (trivial), sehingga A* berperilaku persis seperti Dijkstra tanpa keunggulan apapun, namun dengan overhead komputasi tambahan untuk evaluasi heuristik. Dalam kondisi ini, Dijkstra lebih sederhana dan tepat guna.

4.2.2 Pencarian Jalur ke Semua Tujuan (All-Pairs/Single-Source)

Ketika sistem perlu menghitung jalur dari satu sumber ke semua node lainnya secara serentak (misalnya untuk membangun tabel routing atau analisis keterhubungan), Dijkstra secara alami menghasilkan semua jalur ini dalam satu kali traversal. A* tidak dapat menyamai efisiensi ini karena dirancang untuk satu tujuan; menjalankan A* sebanyak V kali (sekali per tujuan) jauh lebih mahal daripada satu kali eksekusi Dijkstra.

4.2.3 Graf dengan Geometri Tidak Konvensional

Pada graf di mana jalur terpendek harus melakukan detour besar dari arah garis lurus ke tujuan, misalnya karena hambatan geografis besar seperti sungai, tebing, atau dalam kasus navigasi indoor yang memiliki lorong-lorong berliku, heuristik jarak garis lurus menjadi estimasi yang sangat lemah. A* terpaksa mengeksplorasi hampir semua area yang sama dengan Dijkstra, namun dengan overhead evaluasi heuristik tambahan pada setiap node.

4.2.4 Perlu Menemukan Semua Jalur Alternatif

A* standar hanya menemukan satu jalur optimal. Algoritma seperti Yen's K-Shortest Paths yang dibangun di atas Dijkstra lebih cocok ketika sistem perlu menyajikan beberapa alternatif rute kepada pengguna (misalnya rute ke-1, ke-2, dan ke-3 tercepat).

4.2.5 Graf Tanpa Bobot (Unweighted Graph)

Pada graf tanpa bobot atau dengan bobot uniform, BFS menjadi pilihan terbaik karena menghasilkan jalur optimal (minimum hop) dengan kompleksitas $O(V+E)$ yang lebih rendah dari $O((V+E) \log V)$ milik A* dan Dijkstra. Overhead dari heap dan evaluasi heuristik pada A* menjadi beban yang tidak diperlukan.

4.3 Matriks Rekomendasi Pemilihan Algoritma

Karakteristik Sistem / Graf	A*	Dijkstra	BFS	Rekomendasi
Graf berbobot, ada koordinat geografis	✓ YA	✓ YA	✗ TIDAK	A*
Graf tidak berbobot (uniform weights)	~ Parsial	~ Parsial	✓ YA	BFS
Single-pair shortest path	✓ YA	✓ YA	~ Parsial	A*
Single-source semua tujuan	~ Parsial	✓ YA	✗ TIDAK	Dijkstra
Bobot dinamis, kondisi berubah real-time	✓ YA	✗ TIDAK	✗ TIDAK	A*

Karakteristik Sistem / Graf	A*	Dijkstra	BFS	Rekomendasi
Graf tanpa informasi spasial/koordinat	~ Parsial	✓ YA	✓ YA	Dijkstra / BFS
Navigasi jaringan jalan nyata	✓ YA	~ Parsial	✗ TIDAK	A*
Analisis keterhubungan / reachability	✗ TIDAK	~ Parsial	✓ YA	BFS
Rute terpendek + skenario kondisi kompleks	✓ YA	✗ TIDAK	✗ TIDAK	A*
Graf kecil (< 50 node), sembarang tujuan	✓ YA	✓ YA	✓ YA	Ketiganya setara

Tabel 4.1 ,Matriks rekomendasi pemilihan algoritma berdasarkan karakteristik sistem

BAB V KESIMPULAN

5.1 Kesimpulan Analisis

Berdasarkan analisis yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

- Dynamic A* adalah pilihan yang tepat dan optimal untuk sistem navigasi kampus UNIB. Kombinasi heuristik Haversine yang admissible, mekanisme bobot dinamis melalui `calc_cost()`, dan integrasi Scenario Engine menjadikan A* sebagai algoritma yang tidak hanya efisien secara komputasi, tetapi juga adaptif dan dapat dikonfigurasi tanpa mengubah logika inti.
- Fungsi heuristik Haversine memberikan kontribusi signifikan terhadap efisiensi pencarian. Dengan mengestimasi jarak garis lurus berbasis koordinat GPS secara akurat, heuristik ini memungkinkan A* mengeksplorasi 60–80% lebih sedikit node dibandingkan Dijkstra pada skenario pencarian jarak sedang hingga jauh, tanpa mengorbankan optimalitas rute.
- Dijkstra merupakan alternatif yang valid untuk pencarian jalur terpendek pada graf berbobot tanpa kebutuhan adaptasi dinamis. Kompleksitas implementasinya lebih sederhana dan tidak memerlukan informasi koordinat geografis. Namun, Dijkstra tidak cocok untuk sistem yang membutuhkan respons terhadap perubahan kondisi jalan secara real-time seperti yang diimplementasikan dalam UNIB Navigator.
- BFS tidak cocok untuk navigasi pada graf berbobot seperti jaringan jalan kampus UNIB karena mengabaikan bobot edge dan hanya meminimalkan jumlah hop. Rute yang dihasilkan BFS pada jaringan jalan berbobot tidak menjamin jarak fisik atau waktu tempuh minimum. BFS lebih tepat digunakan untuk analisis keterhubungan, pencarian pada graf tidak berbobot, atau pengecekan reachability.
- Dalam skenario no-path (semua jalur terblokir), ketiga algoritma memiliki kompleksitas yang sama, semua node yang dapat dijangkau akan dieksplorasi. Keunggulan A* hilang dalam kondisi ini, menekankan pentingnya mekanisme fallback seperti yang diimplementasikan dalam proyek UNIB (fallback ke skenario Normal dengan peringatan `partial_success`).

5.2 Implikasi untuk Pengembangan Sistem

Temuan analisis ini memiliki beberapa implikasi langsung untuk pengembangan lanjutan UNIB Campus Navigator:

- Implementasi Bidirectional A* dapat dipertimbangkan untuk skenario dengan jumlah node > 1.000 (ekspansi multi-kampus), di mana pencarian dari kedua arah secara bersamaan dapat memangkas ruang pencarian hingga setengahnya.
- Varian LPA* (Lifelong Planning A*) atau D* Lite layak dieksplorasi untuk mendukung pembaruan kondisi jalan inkremental tanpa menjalankan ulang algoritma dari awal, meningkatkan efisiensi pada skenario perubahan kondisi yang sangat sering.

- Pengujian formal dengan 1.000+ kueri berturut-turut diperlukan untuk memvalidasi proyeksi performa empiris dan mengukur P99 execution time sebagai garansi kualitas layanan.
- Kualitas heuristik perlu dievaluasi ulang jika sistem diperluas ke area kampus dengan hambatan geografis signifikan (misalnya jembatan sempit atau lorong yang memaksa detour besar), yang dapat menurunkan akurasi estimasi Haversine.

5.3 Rekomendasi Akhir

Pemilihan Dynamic A* sebagai algoritma inti UNIB Campus Navigator secara keseluruhan merupakan keputusan desain yang tepat dan tervalidasi secara teoritis maupun empiris. Algoritma ini secara unik memenuhi seluruh persyaratan sistem: pencarian single-pair yang efisien, optimalitas rute yang terjamin, adaptabilitas terhadap bobot dinamis, dan dukungan untuk konfigurasi skenario yang modular. Untuk sistem navigasi kampus berbasis koordinat GPS dengan kondisi jalan yang dapat berubah, tidak ada algoritma lain dari ketiga yang dibandingkan dalam dokumen ini yang dapat menandingi kemampuan A* secara komprehensif.